# Review of `XYin.NumberField.FieldExamples`

## 1 Coverage overview

### 1.1 Mathlib `NumberField` tree

The comments record coverage by the current `XYin.NumberField.FieldExamples` folder, not import dependencies. Here "covered" means that the folder contains substantial concrete examples using the corresponding theme or API; "partial" means that only a narrow slice is exercised or some key declarations still contain `sorry`.

**`Mathlib/NumberTheory/NumberField/`**

- `AdeleRing.lean` not yet Adelic objects are not used.
- `Basic.lean` covered Concrete field models and `NumberField` instances throughout.
- `CMField.lean` not yet No CM-field examples yet.
- **`CanonicalEmbedding/`**
  - `Basic.lean` not yet No canonical-embedding examples yet.
  - `ConvexBody.lean` not yet Convex-body geometry is not used.
  - `FundamentalCone.lean` not yet No fundamental-cone coverage.
  - `NormLeOne.lean` not yet No canonical-embedding norm-bound examples.
  - `PolarCoord.lean` not yet No polar-coordinate coverage.
- `ClassNumber.lean` covered Class number 1, class number 2, and class-group structures.
- **`Completion/`**
  - `FinitePlace.lean` not yet Finite completions are not covered.
  - `InfinitePlace.lean` not yet Completion-at-infinity API is not covered.
  - `LiesOverInstances.lean` not yet Lies-over completion instances are not used.
  - `Ramification.lean` not yet Completion-level ramification is not covered.
- **`Cyclotomic/`**
  - `Basic.lean` covered $\zeta_3$, $\zeta_5$, and $\zeta_8$ models, degrees, primitive roots.
  - `Embeddings.lean` partial Automorphisms are covered; embedding-specific API only implicitly.
  - `Galois.lean` covered Cyclotomic Galois groups for $n = 3, 5, 8$.
  - `Ideal.lean` partial Integral closures and splitting appear, but not a full ideal survey.
  - `PID.lean` partial The $\zeta_3$ PID theorem is used; broader PID API is not.
  - `Three.lean` covered $\zeta_3$ units, PID/class number, discriminant, and Galois group.

- `DedekindZeta.lean` not yet Dedekind zeta functions are not covered.
- **`Discriminant/`**
  - `Basic.lean` covered All example fields compute discriminants.
  - `Defs.lean` covered Uses `discr`, `coe_discr`, and basis comparisons.
  - `Different.lean` partial Used for ramified-prime obstructions via the different.
- `EquivReindex.lean` not yet Reindexing equivalences are not singled out.
- `ExistsRamified.lean` not yet Ramification is concrete; this existence API is not used.
- `FinitePlaces.lean` not yet Finite-place API is not covered; ideals are used instead.
- `FractionalIdeal.lean` not yet Fractional ideals are not developed in the examples.
- `House.lean` not yet The house/height-style API is not covered.
- **`Ideal/`**
  - `Asymptotics.lean` not yet Asymptotic ideal counting is not covered.
  - `Basic.lean` covered Prime ideals, absolute norms, ramification, and splitting.
  - `KummerDedekind.lean` not yet Splitting is reproved by residue kernels; this API remains a gap.
- `InfiniteAdeleRing.lean` not yet Infinite adele rings are not used.
- **`InfinitePlace/`**
  - `Basic.lean` covered Signatures and place counts drive the unit-rank examples.
  - `Completion.lean` not yet Infinite-place completions are not covered.
  - `Embeddings.lean` partial Real/complex embedding facts are used for signatures.
  - `Ramification.lean` not yet Infinite-place ramification is not covered.
  - `TotallyRealComplex.lean` covered Totally real/complex statements appear in unit files.
- `Norm.lean` covered Explicit norm forms and unit criteria are central examples.
- `ProductFormula.lean` not yet The product formula is not covered.
- **`Units/`**
  - `Basic.lean` covered Torsion, ranks, and abstract unit decompositions.
  - `DirichletTheorem.lean` partial `fundSystem` APIs are used; concrete generators partly remain `sorry`.
  - `Regulator.lean` partial Regulator statements are present, but several formulas remain `sorry`.

## 1.2 Coverage matrix

Legend:
— means mathematically not applicable;
× means applicable but not explicitly stated in this folder;
△ means stated but with `sorry`;
✓ means stated without `sorry` and easy/formal API driven;
✓ means stated without `sorry` but proof is genuinely involved.

| Statement/Object | $\mathbb{Q}(\sqrt{2})$ | $\mathbb{Q}(\sqrt{5})$ | $\mathbb{Q}(\sqrt{-6})$ | $\mathbb{Q}(\sqrt[3]{3})$ | $\mathbb{Q}(\zeta_3)$ | $\mathbb{Q}(\zeta_5)$ | $\mathbb{Q}(\zeta_8)$ |
|---|---|---|---|---|---|---|---|
| Concrete field model and generator relation | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Alternative presentations/equivalences | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Ring of integers as generated order | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Explicit integral basis | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Discriminant of number field/integral basis | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Norm form and unit criterion | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Concrete splitting of small primes | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| General ramification criterion | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| General splitting or degree-one criterion | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Torsion units / roots of unity | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Signature and Dirichlet rank | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Displayed fundamental unit generates free part | ✓ | △ | — | △ | — | △ | △ |
| Abstract unit group decomposition | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Regulator formula | △ | △ | ✓ | △ | ✓ | △ | △ |
| Class number and class group | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Norm-Euclidean or non-Euclidean statement | ✓ | ✓ | ✓ | △ | ✓ | △ | △ |
| Galois group / automorphism enumeration | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Proper quadratic subfields of quartic cyclotomic fields | — | — | — | — | — | ✓ | ✓ |
| Explicit UFD or non-UFD theorem | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

The file `Common.lean` supplies shared infrastructure rather than one number field.

```
theorem eq_of_le_of_absNorm_eq {R :
    Type*} [CommRing R] [IsDedekindDomain R]
    [Module.Free ℤ R] {I J : Ideal R} (hle : I ≤ J) {n : ℕ}
    (hI : Ideal.absNorm I = n) (hJ : Ideal.absNorm J = n) (hn : n ≠ 0) :
        I = J
```

**Theorem 1.1** (`eq_of_le_of_absNorm_eq`). Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
def torsionPowerHom {G : Type*} [Group G] (n : ℕ) [NeZero n] (g : G) (hg
    : g ^ n = 1) :
    Multiplicative (ZMod n) →* G :=
  MonoidHom.mk' (fun a => g ^ (Multiplicative.toAdd a).val) fun a b =>
      by
    have hpow : ∀ k : ℕ, g ^ k = g ^ (k % n) := by
      intro k
      conv_lhs => rw [← Nat.div_add_mod k n]
      rw [pow_add, pow_mul, hg, one_pow, one_mul]
    calc g ^ (Multiplicative.toAdd (a * b)).val =
        g ^ ((Multiplicative.toAdd (a * b)).val % n) := hpow _
      _ = g ^ (((Multiplicative.toAdd a).val + (Multiplicative.toAdd
          b).val) % n) := by
          rw [toAdd_mul, ZMod.val_add,
              Nat.mod_mod_of_dvd _ (dvd_refl n)]
      _ = g ^ ((Multiplicative.toAdd a).val + (Multiplicative.toAdd
          b).val) := (hpow _).symm
      _ = g ^ (Multiplicative.toAdd a).val * g ^ (Multiplicative.toAdd
          b).val :=
          pow_add _ _ _
```

**Definition 1.2** (`torsionPowerHom`). This declaration introduces a concrete unit or a concrete family of torsion units.

```
def rankOnePowerHom {G : Type*} [Group G] (η : G) :
    Multiplicative (Fin 1 → ℤ) →* G :=
  MonoidHom.mk' (fun f => η ^ (Multiplicative.toAdd f 0)) fun f g => by
    change η ^ ((Multiplicative.toAdd f + Multiplicative.toAdd g) 0) = _
    rw [Pi.add_apply, zpow_add]
```

**Definition 1.3** (rankOnePowerHom)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

```
theorem ramificationIdx_ne_one_of_map_le_sq_of_not_le_pow_succ {R S :
    Type*}
    [CommRing R] [CommRing S] [Algebra R S] {p : Ideal R} {P :
        Ideal S} {n : ℕ}
    (hle2 : Ideal.map (algebraMap R S) p ≤ P ^ 2)
    (hnle : ¬ Ideal.map (algebraMap R S) p ≤ P ^ (n + 1)) :
    Ideal.ramificationIdx p P ≠ 1
```

**Theorem 1.4** (ramificationIdx_ne_one_of_map_le_sq_of_not_le_pow_succ)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow {R :
    Type*}
    [CommRing R] [Algebra ℤ R] [IsDedekindDomain R] [Module.Free ℤ R] {P
        : Ideal R}
    {p n k : ℕ} (hp : p.Prime)
    (habsp : Ideal.absNorm (Ideal.span {((p : ℕ) : R)}) = p ^ n)
    (habsP : Ideal.absNorm P = p ^ k) (hk0 : k ≠ 0) :
    ¬ Ideal.map (algebraMap ℤ R) (Ideal.span {((p : ℕ) :
        ℤ)}) ≤ P ^ (n + 1)
```

**Theorem 1.5** (not_map_span_natCast_le_pow_succ_of_absNorm_span_eq_pow)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 1.6** (Common proof pattern)**.** Mathematically, these are reusable lemmas for norm comparisons, finite cyclic torsion generators, rank-one unit powers, and ramification detection from containments such as $(p) \subseteq P^2$. The proofs are order/ideal algebra: absolute norm divisibility, finite cyclic arithmetic in $\mathbb{Z}/n\mathbb{Z}$, and the definition of ramification index as the largest power containment.

# 2 $\mathbb{Q}(\sqrt{2})$

## 2.1 Basic.lean

```
abbrev K : Type := QuadraticAlgebra ℚ (2 : ℚ) 0
```

**Definition 2.1** (K)**.** This declaration fixes the Lean model for the number field; throughout this section the ring of integers is written directly as $\mathcal{O}$ K.

```
abbrev sqrtTwoRoot : K := QuadraticAlgebra.omega
```

**Definition 2.2** (sqrtTwoRoot)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem sqrtTwoRoot_sq : sqrtTwoRoot ^ 2 = (2 : K)
```

**Theorem 2.3** (sqrtTwoRoot_sq)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
abbrev sqrtTwoPolynomial : ℚ[X] := X ^ 2 - C (2 : ℚ)
```

**Definition 2.4** (sqrtTwoPolynomial)**.** This declaration names the defining polynomial whose root gives the field presentation.

```
def sqrtTwoAdjoinRootEquivQuadratic : KAdjoinRoot ≃ₐ[ℚ] K where
  __ := sqrtTwoAdjoinRootToQuadratic
  invFun := sqrtTwoQuadraticToAdjoinRoot
  left_inv x := by
    have hcomp :
        sqrtTwoQuadraticToAdjoinRoot.comp sqrtTwoAdjoinRootToQuadratic =
          AlgHom.id ℚ KAdjoinRoot := by
      apply AdjoinRoot.algHom_ext
      change sqrtTwoQuadraticToAdjoinRoot
          (sqrtTwoAdjoinRootToQuadratic sqrtTwoAdjoinRoot) =
              sqrtTwoAdjoinRoot
      rw [sqrtTwoAdjoinRootToQuadratic_root,
          sqrtTwoQuadraticToAdjoinRoot_root]
    exact DFunLike.congr_fun hcomp x
  right_inv x := by
    have hcomp :
        sqrtTwoAdjoinRootToQuadratic.comp sqrtTwoQuadraticToAdjoinRoot =
          AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtTwoAdjoinRootToQuadratic
          (sqrtTwoQuadraticToAdjoinRoot sqrtTwoRoot) = sqrtTwoRoot
      rw [sqrtTwoQuadraticToAdjoinRoot_root,
          sqrtTwoAdjoinRootToQuadratic_root]
    exact DFunLike.congr_fun hcomp x
```

**Definition 2.5** (`sqrtTwoAdjoinRootEquivQuadratic`)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

**Theorem 2.6** (`degree_eq_two`)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
def sqrtTwoEquivUnitModel : K ≃ₐ[ℚ] KUnitModel where
  __ := sqrtTwoToUnitModel
  invFun := unitModelToSqrtTwo
  left_inv x := by
    have hcomp :
        unitModelToSqrtTwo.comp sqrtTwoToUnitModel = AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change unitModelToSqrtTwo (sqrtTwoToUnitModel sqrtTwoRoot) =
          sqrtTwoRoot
      rw [sqrtTwoToUnitModel_root]
      simp only [map_sub, map_one, unitModelToSqrtTwo_root]
      ring
    change (unitModelToSqrtTwo.comp sqrtTwoToUnitModel) x =
        (AlgHom.id ℚ K) x
    rw [hcomp]
  right_inv x := by
    have hcomp :
        sqrtTwoToUnitModel.comp unitModelToSqrtTwo =
            AlgHom.id ℚ KUnitModel := by
      apply AdjoinRoot.algHom_ext
      change sqrtTwoToUnitModel (unitModelToSqrtTwo sqrtTwoUnitRoot) =
          sqrtTwoUnitRoot
      rw [unitModelToSqrtTwo_root]
      simp only [map_add, map_one, sqrtTwoToUnitModel_root]
      ring
    change (sqrtTwoToUnitModel.comp unitModelToSqrtTwo) x =
      (AlgHom.id ℚ KUnitModel) x
    rw [hcomp]
```

**Definition 2.7** (`sqrtTwoEquivUnitModel`)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

**Remark 2.8** (Subsection note)**.** This builds $\mathbb{Q}(\sqrt{2})$, proves $\sqrt{2}^2 = 2$, identifies the quadratic polynomial, gives the `AdjoinRoot` model, and also the unit-root model $u^2 - 2u - 1 = 0$. The proof uses the existing quadratic algebra API and two mutually inverse algebra homomorphisms.

## 2.2 RingOfIntegers.lean

```
theorem sqrtTwoRoot_isIntegral : IsIntegral ℤ sqrtTwoRoot
```

**Theorem 2.9** (sqrtTwoRoot_isIntegral)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
abbrev sqrtTwoInteger : 𝓞 K :=
  ⟨sqrtTwoRoot, sqrtTwoRoot_isIntegral⟩
```

**Definition 2.10** (sqrtTwoInteger)**.** This is the algebraic integer $\sqrt{2}$ as an element of $\mathcal{O}_K$; the code now uses $\boldsymbol{\mathcal{O}}$ K directly rather than a separate ring-of-integers alias.

```
theorem sqrtTwo_minpoly_int :
    minpoly ℤ sqrtTwoRoot = X ^ 2 - C (2 : ℤ)
```

**Theorem 2.11** (sqrtTwo_minpoly_int)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem isIntegralClosure_adjoin_sqrtTwoRoot :
    IsIntegralClosure (Algebra.adjoin ℤ ({sqrtTwoRoot} : Set K)) ℤ K
```

**Theorem 2.12** (isIntegralClosure_adjoin_sqrtTwoRoot)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
abbrev sqrtTwoExplicitIntegralBasis : Basis (Fin 2) ℤ (𝓞 K) :=
  sqrtTwoExplicitIntegralPowerBasis.basis.reindex
    (finCongr sqrtTwoExplicitIntegralPowerBasis_dim)
```

**Definition 2.13** (sqrtTwoExplicitIntegralBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
theorem sqrtTwo_numberField_discr : NumberField.discr K = 8
```

**Theorem 2.14** (sqrtTwo_numberField_discr)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
example
    (P : Matrix (Fin sqrtTwoPowerBasis.dim) (Fin sqrtTwoPowerBasis.dim)
        ℚ) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
        sqrtTwoPowerBasis.basis) =
      P.det ^ 2 * Algebra.discr ℚ sqrtTwoPowerBasis.basis
```

**Theorem 2.15** (`example`)**.** Mathematically, applying a rational matrix $P$ to the explicit power basis scales the discriminant by $\det(P)^2$.

*Proof sketch.* The Lean proof applies the change-of-basis formula
`Algebra.discr_of_matrix_mulVec`. □

```
example
    (P : Matrix (Fin sqrtTwoPowerBasis.dim) (Fin sqrtTwoPowerBasis.dim)
        ℚ)
    (hdet : P.det = 1 ∨ P.det = -1) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
        sqrtTwoPowerBasis.basis) =
      (8 : ℚ)
```

**Theorem 2.16** (`example`)**.** Mathematically, a unimodular change of basis preserves the discriminant, so this transformed $\mathbb{Q}$-basis still has discriminant 8.

*Proof sketch.* The Lean proof combines the matrix discriminant formula,
`sqrtTwoPowerBasis_discr`, and $\det(P) = \pm 1$. □

**Remark 2.17** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\sqrt{2}]$, with basis $1, \sqrt{2}$ and discriminant 8. The proof is via integrality of the root, Eisenstein/minimal polynomial information, and discriminant computation from the power basis.

## 2.3 `NormForm.lean`

```
theorem norm_eq_normForm (x : K) :
    Algebra.norm ℚ x = x.re ^ 2 - 2 * x.im ^ 2
```

**Theorem 2.18** (`norm_eq_normForm`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝒪 K) + y • sqrtTwoInteger) = x ^ 2 - 2 * y
        ^ 2
```

**Theorem 2.19** (`intNorm_eq_normForm`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • sqrtTwoInteger) ↔
      x ^ 2 - 2 * y ^ 2 = 1 ∨ x ^ 2 - 2 * y ^ 2 = -1
```

**Theorem 2.20** (`isUnit_iff_normForm`)**.** Mathematically, this converts the unit condition into the corresponding norm-form equation.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 2.21** (Subsection note)**.** This is the classical norm form $x^2 - 2y^2$. The Lean proof expands the algebra norm in the quadratic basis and then uses that an algebraic integer is a unit exactly when its integer norm is $\pm 1$.

## 2.4 `PrimeSplitting.lean`

```
theorem span_two_eq : Ideal.span {(2 :
    𝒪 K)} = Ideal.span {sqrtTwoInteger} ^ 2
```

**Theorem 2.22** (`span_two_eq`)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} =
      Ideal.span {3 + sqrtTwoInteger} * Ideal.span {3 - sqrtTwoInteger}
```

**Theorem 2.23** (`span_seven_eq`)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_three : (Ideal.span {(3 : 𝒪 K)}).IsPrime
```

**Theorem 2.24** (isPrime_span_three)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem ramified_iff_eq_two (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p :
        𝒪 K)} ≤ P ^ 2) ↔ p = 2
```

**Theorem 2.25** (ramified_iff_eq_two)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem splits_iff_pm_one_mod_eight (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2)
    :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 8 = 1 ∨ p % 8 = 7
```

**Theorem 2.26** (splits_iff_pm_one_mod_eight)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 2.27** (Subsection note)**.** Mathematically, 2 ramifies, 7 splits, 3 is inert, the only ramified prime is 2, and odd primes split exactly for $p \equiv \pm 1 \pmod 8$. The proof combines explicit ideal equalities, norm computations, primality from absolute norms, and quadratic residue arithmetic modulo 8.

## 2.5 Units.lean

```
abbrev sqrtTwoFundamentalUnit : (𝒪 K)ˣ where
  val := sqrtTwoInteger + 1
  inv := sqrtTwoInteger - 1
  val_inv := by
    apply Subtype.ext
    calc
      (sqrtTwoRoot + 1) * (sqrtTwoRoot - 1) = sqrtTwoRoot ^ 2 - 1 :=
          by ring
      _ = (1 : K) := by rw [sqrtTwoRoot_sq]; ring
  inv_val := by
    apply Subtype.ext
```

```
    calc
      (sqrtTwoRoot - 1) * (sqrtTwoRoot + 1) = sqrtTwoRoot ^ 2 - 1 :=
          by ring
      _ = (1 : K) := by rw [sqrtTwoRoot_sq]; ring
```

**Definition 2.28** (`sqrtTwoFundamentalUnit`)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem sqrtTwoFundamentalUnit_generates_unitGroup (u : (𝓞 K)ˣ) :
    ∃ n : ℤ, u = sqrtTwoFundamentalUnit ^ n ∨
      u = -(sqrtTwoFundamentalUnit ^ n)
```

**Theorem 2.29** (`sqrtTwoFundamentalUnit_generates_unitGroup`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem torsionOrder_eq_two : Units.torsionOrder K = 2
```

**Theorem 2.30** (`torsionOrder_eq_two`)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_one : Units.rank K = 1
```

**Theorem 2.31** (`unitRank_eq_one`)**.** Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem regulator_eq : Units.regulator K = Real.log (1 + Real.sqrt 2)
```

**Theorem 2.32** (`regulator_eq`)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

```
theorem unitGroupStructure :
    Nonempty ((𝓞 K)ˣ ≃* (Multiplicative (ZMod 2) ×
        Multiplicative (Fin 1 → ℤ)))
```

**Theorem 2.33** (`unitGroupStructure`)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 2.34** (Subsection note)**.** The unit $1+\sqrt{2}$ generates all units up to sign, via mathlib's Pell API for $x^2 - 2y^2 = 1$. The torsion is $\{\pm 1\}$, the rank is 1, and the abstract group is $\mu_2 \times \mathbb{Z}$. The regulator formula is stated but still has `sorry`.

## 2.6 `ClassGroup.lean`

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝓞 K)
```

**Theorem 2.35** (`isPrincipalIdealRing`)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝓞 K)
```

**Theorem 2.36** (`uniqueFactorizationMonoid`)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 2.37** (`classNumber_eq_one`)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝓞 K) ≃* Multiplicative (ZMod 1))
```

**Theorem 2.38** (`classGroupStructure`)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 2.39** (Subsection note)**.** The ring of integers is a PID and has trivial class group. The formal proof uses the norm-Euclidean statement and the equivalence between class number one and principal ideal ring.

## 2.7 Euclidean.lean

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 2.40** (normEuclidean)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* The proof chooses a nearest integral-coordinate quotient and bounds the norm of the remainder; in the unfinished higher-degree cases this is exactly the missing geometry-of-numbers step. □

**Remark 2.41** (Subsection note)**.** This formalizes the norm-Euclidean algorithm: divide $a/b$, round both coordinates in the $1, \sqrt{2}$ basis, and bound the norm of the remainder by the rectangle estimate $\leq 1/2$.

## 2.8 Galois.lean

```
def sqrtTwoConj : K ≃ₐ[ℚ] K where
  __ := sqrtTwoConjHom
  invFun := sqrtTwoConjHom
  left_inv x := by
    have hcomp : sqrtTwoConjHom.comp sqrtTwoConjHom = AlgHom.id ℚ K :=
        by
      apply QuadraticAlgebra.algHom_ext
      change sqrtTwoConjHom (sqrtTwoConjHom sqrtTwoRoot) = sqrtTwoRoot
      rw [sqrtTwoConjHom_root]
      simp only [map_neg, sqrtTwoConjHom_root, neg_neg]
    change (sqrtTwoConjHom.comp sqrtTwoConjHom) x = (AlgHom.id ℚ K) x
    rw [hcomp]
  right_inv x := by
    have hcomp : sqrtTwoConjHom.comp sqrtTwoConjHom = AlgHom.id ℚ K :=
        by
      apply QuadraticAlgebra.algHom_ext
      change sqrtTwoConjHom (sqrtTwoConjHom sqrtTwoRoot) = sqrtTwoRoot
      rw [sqrtTwoConjHom_root]
      simp only [map_neg, sqrtTwoConjHom_root, neg_neg]
    change (sqrtTwoConjHom.comp sqrtTwoConjHom) x = (AlgHom.id ℚ K) x
    rw [hcomp]
```

**Definition 2.42** (sqrtTwoConj)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem sqrtTwoConj_root : sqrtTwoConj sqrtTwoRoot = -sqrtTwoRoot
```

**Theorem 2.43** (`sqrtTwoConj_root`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem isGalois : IsGalois ℚ K
```

**Theorem 2.44** (`isGalois`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroupStructure :
    Nonempty (Gal(K/ℚ) ≃* Multiplicative (ZMod 2))
```

**Theorem 2.45** (`galoisGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 2.46** (Subsection note)**.** The nontrivial automorphism sends $\sqrt{2} \mapsto -\sqrt{2}$. Every automorphism is either identity or conjugation, so the Galois group is $C_2$.

### 2.9 Summary for $\mathbb{Q}(\sqrt{2})$

Easy: field construction, integral basis, norm form, class group once Euclideanity is known. Hard but completed: Pell-based concrete unit generation, prime splitting criterion modulo 8, norm-Euclidean rounding. Not completed: the regulator formula.

# 3 $\mathbb{Q}(\sqrt{5})$

## 3.1 `Basic.lean`

```
abbrev K : Type := QuadraticAlgebra ℚ (1 : ℚ) (1 : ℚ)
```

**Definition 3.1** (`K`)**.** This declaration fixes the Lean model for the number field; the ring of integers is written directly as $\mathcal{O}$ `K`.

```
abbrev goldenRoot : K := QuadraticAlgebra.omega
```

**Definition 3.2** (`goldenRoot`)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem goldenRoot_sq : goldenRoot ^ 2 = goldenRoot + 1
```

**Theorem 3.3** (goldenRoot_sq)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
abbrev sqrtFiveRoot : K := 2 * goldenRoot - 1
```

**Definition 3.4** (sqrtFiveRoot)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem sqrtFiveRoot_sq : sqrtFiveRoot ^ 2 = (5 : K)
```

**Theorem 3.5** (sqrtFiveRoot_sq)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
def sqrtFiveRadicalEquivGolden : RadicalModel ≃ₐ[ℚ] K where
  __ := sqrtFiveRadicalToGolden
  invFun := goldenToSqrtFiveRadical
  left_inv x := by
    have hcomp :
        goldenToSqrtFiveRadical.comp sqrtFiveRadicalToGolden =
          AlgHom.id ℚ RadicalModel := by
      apply QuadraticAlgebra.algHom_ext
      change goldenToSqrtFiveRadical
          (sqrtFiveRadicalToGolden sqrtFiveRadicalRoot) =
              sqrtFiveRadicalRoot
      rw [sqrtFiveRadicalToGolden_root, sqrtFiveRoot, map_sub, map_mul,
          map_ofNat,
        goldenToSqrtFiveRadical_root, map_one]
      ring
    exact DFunLike.congr_fun hcomp x
  right_inv x := by
    have hcomp :
        sqrtFiveRadicalToGolden.comp goldenToSqrtFiveRadical =
          AlgHom.id ℚ K := by
      apply QuadraticAlgebra.algHom_ext
      change sqrtFiveRadicalToGolden
          (goldenToSqrtFiveRadical goldenRoot) = goldenRoot
```

```
    rw [goldenToSqrtFiveRadical_root, map_div₀, map_add, map_one,
      sqrtFiveRadicalToGolden_root]
    have hmap_two : sqrtFiveRadicalToGolden (2 : RadicalModel) = (2 :
        K) := by
      simpa using sqrtFiveRadicalToGolden.commutes (2 : ℚ)
    rw [hmap_two]
    exact goldenRoot_eq_half_sqrtFiveRoot_add_one.symm
  exact DFunLike.congr_fun hcomp x
```

**Definition 3.6** (`sqrtFiveRadicalEquivGolden`)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

**Remark 3.7** (Subsection note)**.** The main model is $\mathbb{Q}(\varphi)$, $\varphi^2 = \varphi + 1$, together with the radical model $\sqrt{5} = 2\varphi - 1$. The proof builds explicit algebra equivalences between the golden-ratio and radical presentations.

## 3.2 `RingOfIntegers.lean`

```
abbrev GoldenOrder : Type := QuadraticAlgebra ℤ (1 : ℤ) (1 : ℤ)
```

**Definition 3.8** (`GoldenOrder`)**.** This is the concrete order $\mathbb{Z}[\varphi]$ used to prove integral closure; the abstract ring of integers is still written as $\boldsymbol{\mathcal{O}}$ `K`.

```
theorem integral_mem_goldenOrderToField_range {x : K}
    (hx : IsIntegral ℤ x) : ∃ y :
        GoldenOrder, algebraMap GoldenOrder K y = x
```

**Theorem 3.9** (`integral_mem_goldenOrderToField_range`)**.** Mathematically, this proves that the displayed algebraic element is integral over $\mathbb{Z}$.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
instance : IsIntegralClosure GoldenOrder ℤ K
```

**Definition 3.10** (`instance`)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem ringOfIntegers_eq :
    integralClosure ℤ K = concreteRingOfIntegers
```

**Theorem 3.11** (`ringOfIntegers_eq`)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem adjoin_sqrtFive_ne_integralClosure :
    Algebra.adjoin ℤ ({sqrtFiveRoot} : Set K) ≠ integralClosure ℤ K
```

**Theorem 3.12** (adjoin_sqrtFive_ne_integralClosure)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem numberField_discr_eq : NumberField.discr K = 5
```

**Theorem 3.13** (numberField_discr_eq)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 3.14** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\varphi]$, not $\mathbb{Z}[\sqrt{5}]$, with basis $1, \varphi$ and discriminant 5. The hard step is proving integral elements land in the golden order, using trace/norm congruences.

## 3.3 NormForm.lean

```
noncomputable def goldenInt0 : 𝒪 K :=
    ⟨goldenRoot, goldenRoot_isIntegral⟩
```

**Definition 3.15** (goldenInt0)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

```
theorem norm_eq_normForm (x y : ℚ) :
    Algebra.norm ℚ (x • 1 + y • goldenRoot) = x ^ 2 + x * y - y ^ 2
```

**Theorem 3.16** (norm_eq_normForm)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝒪 K) + y • goldenInt0) ↔
      x ^ 2 + x * y - y ^ 2 = 1 ∨ x ^ 2 + x * y - y ^ 2 = -1
```

**Theorem 3.17** (isUnit_iff_normForm)**.** Mathematically, this converts the unit condition into the corresponding norm-form equation.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 3.18** (Subsection note)**.** The norm form is $x^2 + xy - y^2$. Units are exactly integral pairs with norm $\pm 1$.

## 3.4 PrimeSplitting.lean

```
theorem span_five_eq :
    Ideal.span {(5 : 𝒪 K)} = Ideal.span {sqrtFiveInt0} ^ 2
```

**Theorem 3.19** (span_five_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝒪 K)}).IsPrime
```

**Theorem 3.20** (isPrime_span_two)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_three : (Ideal.span {(3 : 𝒪 K)}).IsPrime
```

**Theorem 3.21** (isPrime_span_three)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_eleven_eq :
    Ideal.span {(11 : 𝒪 K)} =
      Ideal.span {3 + goldenInt0} * Ideal.span {4 - goldenInt0}
```

**Theorem 3.22** (span_eleven_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem ramified_iff_eq_five (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧
      Ideal.span {(p : 𝒪 K)} ≤ P ^ 2) ↔ p = 5
```

**Theorem 3.23** (ramified_iff_eq_five)**.** Mathematically, this classifies exactly which rational primes ramify in the field.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem splits_iff_pm_one_mod_five (p : ℕ) (hp : p.Prime) (hp5 : p ≠ 5)
    :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 5 = 1 ∨ p % 5 = 4
```

**Theorem 3.24** (splits_iff_pm_one_mod_five)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 3.25** (Subsection note)**.** Here 5 ramifies, 2 and 3 are inert, 11 splits, ramification occurs only at 5, and splitting away from 5 is $p \equiv \pm 1 \pmod 5$. The proof uses the explicit norm form and quadratic residue computations.

## 3.5 Units.lean

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := goldenInt0
  inv := goldenInt0 - 1
  val_inv := by
    rw [show goldenInt0 * (goldenInt0 - 1) = goldenInt0 ^ 2 - goldenInt0
        by ring]
    rw [goldenInt0_sq]
    ring
  inv_val := by
    rw [show (goldenInt0 - 1) * goldenInt0 = goldenInt0 ^ 2 - goldenInt0
        by ring]
    rw [goldenInt0_sq]
    ring
```

**Definition 3.26** (fundamentalUnit)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem torsionOrder_eq_two : Units.torsionOrder K = 2
```

**Theorem 3.27** (torsionOrder_eq_two)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_one : Units.rank K = 1
```

**Theorem 3.28** (unitRank_eq_one)**.** Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem fundamentalUnit_generates (u : (𝒪 K)ˣ) :
    ∃ n : ℤ, u = fundamentalUnit ^ n ∨ u = -(fundamentalUnit ^ n)
```

**Theorem 3.29** (`fundamentalUnit_generates`)**.** Mathematically, this states that the displayed unit generates the free part of the unit group, up to torsion.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

```
theorem regulator_eq :
    Units.regulator K = Real.log ((1 + Real.sqrt 5) / 2)
```

**Theorem 3.30** (`regulator_eq`)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 3.31** (Subsection note)**.** The displayed unit is $\varphi$. Torsion and rank are proved; the concrete statement that every unit is a signed power of $\varphi$ is stated with `sorry`, as is the regulator. The missing bridge is the Pell/Fibonacci equation $a^2 - 5b^2 = \pm 4$.

## 3.6 `ClassGroup.lean`

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 3.32** (`classNumber_eq_one`)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝒪 K)
```

**Theorem 3.33** (`isPrincipalIdealRing`)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝒪 K)
```

**Theorem 3.34** (`uniqueFactorizationMonoid`)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 1))
```

**Theorem 3.35** (`classGroupStructure`)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 3.36** (Subsection note)**.** Class number one follows by a Minkowski-bound argument and the small-prime splitting already developed.

## 3.7 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 3.37** (`normEuclidean`)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* The proof chooses a nearest integral-coordinate quotient and bounds the norm of the remainder; in the unfinished higher-degree cases this is exactly the missing geometry-of-numbers step. □

**Remark 3.38** (Subsection note)**.** The Euclidean proof rounds coordinates in the $1, \varphi$ basis and bounds $|u^2 + uv - v^2| \leq 3/4$ for the fractional remainder.

## 3.8 `Galois.lean`

```
noncomputable def conjugation : K ≃ₐ[ℚ] K where
  __ := conjHom
  invFun := conjHom
  left_inv x := DFunLike.congr_fun conjHom_comp x
  right_inv x := DFunLike.congr_fun conjHom_comp x
```

**Definition 3.39** (`conjugation`)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem conjugation_apply_goldenRoot :
    conjugation goldenRoot = 1 - goldenRoot
```

**Theorem 3.40** (`conjugation_apply_goldenRoot`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem isGalois : IsGalois ℚ K
```

**Theorem 3.41** (`isGalois`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroupStructure :
    Nonempty ((K ≃ₐ[ℚ] K) ≃* Multiplicative (ZMod 2))
```

**Theorem 3.42** (`galoisGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 3.43** (Subsection note)**.** The conjugation is $\varphi \mapsto 1 - \varphi$, equivalently $\sqrt{5} \mapsto -\sqrt{5}$, and the Galois group is $C_2$.

### 3.9 Summary for $\mathbb{Q}(\sqrt{5})$

Easy: field equivalences, Galois group, norm form. Hard but completed: proving $\mathbb{Z}[\varphi]$ is the full ring of integers, splitting modulo 5, class number one, and norm-Euclideanity. Missing: concrete generation of all units by $\varphi$ and the regulator formula.

## 4 $\mathbb{Q}(\sqrt{-6})$

### 4.1 `Basic.lean`

```
abbrev K : Type := QuadraticAlgebra ℚ (-6 : ℚ) 0
```

**Definition 4.1** (`K`)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
abbrev θ : K := QuadraticAlgebra.omega
```

**Definition 4.2** (`θ`)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem theta_sq : θ ^ 2 = -6
```

**Theorem 4.3** (theta_sq)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

**Theorem 4.4** (degree_eq_two)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
noncomputable def adjoinRootEquivQuadratic : KAdjoinRoot ≃ₐ[ℚ] K := by
  let f : KAdjoinRoot →ₐ[ℚ] K :=
    AdjoinRoot.liftAlgHom sqrtNegSixPolynomial (Algebra.ofId ℚ K) θ
      sqrtNegSixPolynomial_aeval_theta
  let g : K →ₐ[ℚ] KAdjoinRoot :=
    QuadraticAlgebra.lift ⟨thetaAdjoinRoot, thetaAdjoinRoot_mul_self⟩
  have hfroot : f thetaAdjoinRoot = θ := by
    change (AdjoinRoot.liftAlgHom sqrtNegSixPolynomial (Algebra.ofId ℚ
        K) θ
        sqrtNegSixPolynomial_aeval_theta)
            (AdjoinRoot.root sqrtNegSixPolynomial) = θ
    exact AdjoinRoot.liftAlgHom_root sqrtNegSixPolynomial (Algebra.ofId
        ℚ K) θ
      sqrtNegSixPolynomial_aeval_theta
  have hgroot : g θ = thetaAdjoinRoot := by
    change (QuadraticAlgebra.omega : K).re • (1 : KAdjoinRoot) +
      (QuadraticAlgebra.omega :
          K).im • thetaAdjoinRoot = thetaAdjoinRoot
    simp
  refine AlgEquiv.ofAlgHom f g ?_ ?_
  · apply QuadraticAlgebra.algHom_ext
    change f (g θ) = θ
    rw [hgroot]
    exact hfroot
  · apply AdjoinRoot.algHom_ext
    change g (f thetaAdjoinRoot) = thetaAdjoinRoot
    rw [hfroot]
    exact hgroot
```

**Definition 4.5** (`adjoinRootEquivQuadratic`)**.** This declaration names the distinguished algebraic generator of the field.

**Remark 4.6** (Subsection note)**.** This is the imaginary quadratic field generated by $\theta^2 = -6$. The model is equivalent to the `AdjoinRoot` model for $X^2 + 6$.

## 4.2 `RingOfIntegers.lean`

```
theorem theta_integral : θ ∈ integralClosure ℤ K
```

**Theorem 4.7** (`theta_integral`)**.** Mathematically, this proves that the displayed algebraic element is integral over $\mathbb{Z}$.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
noncomputable def thetaInt : 𝒪 K := ⟨θ, theta_integral⟩
```

**Definition 4.8** (`thetaInt`)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem sqrtNegSix_minpoly_int_isEisensteinAt_two :
    (minpoly ℤ θ).IsEisensteinAt 𝒫₂
```

**Theorem 4.9** (`sqrtNegSix_minpoly_int_isEisensteinAt_two`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem sqrtNegSix_minpoly_int_isEisensteinAt_three :
    (minpoly ℤ θ).IsEisensteinAt 𝒫₃
```

**Theorem 4.10** (`sqrtNegSix_minpoly_int_isEisensteinAt_three`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

**Theorem 4.11** (`ringOfIntegers_eq`)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem numberField_discr_eq : NumberField.discr K = (-24 : ℤ)
```

**Theorem 4.12** (`numberField_discr_eq`)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 4.13** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\sqrt{-6}]$, with basis $1, \theta$ and discriminant $-24$. Eisenstein information at 2 and 3 supports the integral basis/discriminant calculations.

## 4.3 NormForm.lean

```
theorem norm_eq_normForm (x y : ℚ) :
    Algebra.norm ℚ (x • (1 : K) + y • θ) = x ^ 2 + 6 * y ^ 2
```

**Theorem 4.14** (`norm_eq_normForm`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝓞 K) + y • thetaInt) ↔ x ^ 2 + 6 * y ^ 2 = 1
```

**Theorem 4.15** (`isUnit_iff_normForm`)**.** Mathematically, this converts the unit condition into the corresponding norm-form equation.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem intNorm_ne_two (z : 𝓞 K) : Algebra.norm ℤ z ≠ 2
```

**Theorem 4.16** (`intNorm_ne_two`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem intNorm_ne_three (z : 𝓞 K) : Algebra.norm ℤ z ≠ 3
```

**Theorem 4.17** (`intNorm_ne_three`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

**Remark 4.18** (Subsection note)**.** The positive norm form is $x^2 + 6y^2$. It proves the only units are $\pm 1$, and it rules out elements of norms $2, 3, 5$, which is crucial for nonprincipal ideals.

## 4.4 PrimeSplitting.lean

```
def primeAboveTwo : Ideal (𝒪 K) := Ideal.span {(2 : 𝒪 K), thetaInt}
```

**Definition 4.19** (primeAboveTwo)**.** This declaration introduces a concrete ideal used to describe splitting or ideal classes.

```
def primeAboveThree : Ideal (𝒪 K) := Ideal.span {(3 : 𝒪 K), thetaInt}
```

**Definition 4.20** (primeAboveThree)**.** This declaration introduces a concrete ideal used to describe splitting or ideal classes.

```
theorem span_two_eq : Ideal.span {(2 : 𝒪 K)} = primeAboveTwo ^ 2
```

**Theorem 4.21** (span_two_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_three_eq : Ideal.span {(3 : 𝒪 K)} = primeAboveThree ^ 2
```

**Theorem 4.22** (span_three_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem not_isPrincipal_primeAboveTwo :
    ¬ Submodule.IsPrincipal primeAboveTwo
```

**Theorem 4.23** (not_isPrincipal_primeAboveTwo)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} = Ideal.span {1 + thetaInt} * Ideal.span {1 -
        thetaInt}
```

**Theorem 4.24** (span_seven_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem splits_iff_mod_twentyfour (p : ℕ) (hp : p.Prime)
    (hp2 : p ≠ 2) (hp3 : p ≠ 3) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔
        p % 24 = 1 ∨ p % 24 = 5 ∨ p % 24 = 7 ∨ p % 24 = 11
```

**Theorem 4.25** (splits_iff_mod_twentyfour)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem ramified_iff (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p :
        𝒪 K)} ≤ P ^ 2) ↔ p = 2 ∨ p = 3
```

**Theorem 4.26** (ramified_iff)**.** Mathematically, this classifies exactly which rational primes ramify in the field.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 4.27** (Subsection note)**.** The primes 2 and 3 ramify, their ramified prime ideals are nonprincipal, 7 and 5 split, 13 is inert, splitting is classified modulo 24, and ramification occurs exactly at $2, 3$. The proof is an explicit ideal/norm analysis.

## 4.5 Units.lean

```
theorem isTotallyComplex : NumberField.IsTotallyComplex K
```

**Theorem 4.28** (isTotallyComplex)**.** Mathematically, this computes the archimedean signature of the field.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem torsionOrder_eq_two : NumberField.Units.torsionOrder K = 2
```

**Theorem 4.29** (torsionOrder_eq_two)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unit_eq_one_or_neg_one (u : (𝒪 K)ˣ) : u = 1 ∨ u = -1
```

**Theorem 4.30** (unit_eq_one_or_neg_one)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_zero : NumberField.Units.rank K = 0
```

**Theorem 4.31** (unitRank_eq_zero)**.** Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem regulator_eq_one : NumberField.Units.regulator K = 1
```

**Theorem 4.32** (regulator_eq_one)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 4.33** (Subsection note)**.** The field has signature $(0, 1)$, rank 0, units $\{\pm 1\}$, and regulator 1.

## 4.6 ClassGroup.lean

```
noncomputable def ramifiedIdealOverTwo : (Ideal (𝓞 K))⁰ :=
  ⟨primeAboveTwo, by
    rw [← Ideal.absNorm_ne_zero_iff_mem_nonZeroDivisors]
    rw [absNorm_primeAboveTwo]
    norm_num⟩
```

**Definition 4.34** (ramifiedIdealOverTwo)**.** This declaration introduces a concrete ideal used to describe splitting or ideal classes.

```
theorem classGroup_mk0_ramified_ne_one :
    ClassGroup.mk0 ramifiedIdealOverTwo ≠ 1
```

**Theorem 4.35** (classGroup_mk0_ramified_ne_one)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem classGroup_mk0_ramified_sq_eq_one :
    ClassGroup.mk0 ramifiedIdealOverTwo ^ 2 = 1
```

**Theorem 4.36** (`classGroup_mk0_ramified_sq_eq_one`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem classNumber_eq_two : NumberField.classNumber K = 2
```

**Theorem 4.37** (`classNumber_eq_two`)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝒪 K) ≃* Multiplicative (ZMod 2))
```

**Theorem 4.38** (`classGroupStructure`)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 4.39** (Subsection note)**.** The class group is $C_2$, generated by the ramified prime over 2. The proof uses Minkowski's bound, identifies all possible small-norm ideal classes, and proves the ramified class is nontrivial but has square one.

## 4.7 `Euclidean.lean`

```
theorem not_isPrincipalIdealRing : ¬ IsPrincipalIdealRing (𝒪 K)
```

**Theorem 4.40** (`not_isPrincipalIdealRing`)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem not_normEuclidean :
    ¬ ∀ a b : 𝒪 K, b ≠ 0 →
      ∃ q r : 𝒪 K, a = b * q + r ∧
        (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 4.41** (`not_normEuclidean`)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* The proof chooses a nearest integral-coordinate quotient and bounds the norm of the remainder; in the unfinished higher-degree cases this is exactly the missing geometry-of-numbers step. □

**Remark 4.42** (Subsection note)**.** Since the ring is not a PID, it is not norm-Euclidean. The second proof also gives a direct obstruction by attempting to divide $\theta$ by 2.

## 4.8 `UniqueFactorization.lean`

```
theorem irreducible_two : Irreducible (2 : 𝒪 K)
```

**Theorem 4.43** (`irreducible_two`)**.** Mathematically, this is part of the explicit unique-factorization analysis.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem irreducible_three : Irreducible (3 : 𝒪 K)
```

**Theorem 4.44** (`irreducible_three`)**.** Mathematically, this is part of the explicit unique-factorization analysis.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem irreducible_theta : Irreducible thetaInt
```

**Theorem 4.45** (`irreducible_theta`)**.** Mathematically, this is part of the explicit unique-factorization analysis.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem two_not_prime : ¬ Prime (2 : 𝒪 K)
```

**Theorem 4.46** (`two_not_prime`)**.** Mathematically, this is part of the explicit unique-factorization analysis.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem not_uniqueFactorizationMonoid :
    ¬ UniqueFactorizationMonoid (𝒪 K)
```

**Theorem 4.47** (`not_uniqueFactorizationMonoid`)**.** Mathematically, this is part of the explicit unique-factorization analysis.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

**Remark 4.48** (Subsection note)**.** This gives an explicit failure of unique factorization: $6 = -\theta^2$, while 2, 3, and $\theta$ are irreducible and 2 is not prime.

## 4.9 `Galois.lean`

```
noncomputable def conjugation : K ≃ₐ[ℚ] K :=
  AlgEquiv.ofAlgHom conjugationHom conjugationHom
    conjugationHom_conjugationHom conjugationHom_conjugationHom
```

**Definition 4.49** (`conjugation`)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem conjugation_apply_theta : conjugation θ = -θ
```

**Theorem 4.50** (`conjugation_apply_theta`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem isGalois : IsGalois ℚ K
```

**Theorem 4.51** (`isGalois`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroupStructure :
    Nonempty ((K ≃ₐ[ℚ] K) ≃* Multiplicative (ZMod 2))
```

**Theorem 4.52** (`galoisGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 4.53** (Subsection note)**.** The Galois group is $C_2$, generated by complex conjugation.

## 4.10 Summary for $\mathbb{Q}(\sqrt{-6})$

Easy: units and Galois structure. Hard but completed: class number 2, explicit nonprincipal ramified ideals, splitting modulo 24, and non-UFD. Nothing important in this folder for this field remains behind `sorry`.

# 5 $\mathbb{Q}(\sqrt[3]{3})$

## 5.1 Basic.lean

```
abbrev cubePolynomial : ℚ[X] := X ^ 3 - C (3 : ℚ)
```

**Definition 5.1** (cubePolynomial)**.** This declaration names the defining polynomial whose root gives the field presentation.

```
abbrev K : Type := AdjoinRoot cubePolynomial
```

**Definition 5.2** (K)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
abbrev cubeRootThree : K := AdjoinRoot.root cubePolynomial
```

**Definition 5.3** (cubeRootThree)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem cubeRootThree_cube : cubeRootThree ^ 3 = 3
```

**Theorem 5.4** (cubeRootThree_cube)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem degree_eq_three : Module.finrank ℚ K = 3
```

**Theorem 5.5** (degree_eq_three)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem cubeRootThree_minpoly_rat :
    minpoly ℚ cubeRootThree = cubePolynomial
```

**Theorem 5.6** (cubeRootThree_minpoly_rat)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

**Remark 5.7** (Subsection note)**.** The field is the pure cubic field generated by $\theta^3 = 3$. Irreducibility gives degree 3, and the AdjoinRoot power basis supplies the basic structure.

## 5.2 RingOfIntegers.lean

```
theorem cubeRootThree_integral : cubeRootThree ∈ integralClosure ℤ K
```

**Theorem 5.8** (cubeRootThree_integral)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem norm_eq_normForm (x y z : ℚ) :
    Algebra.norm ℚ (x + yθ + zθ^2) =
      x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z
```

**Theorem 5.9** (norm_eq_normForm)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem integralClosure_eq_adjoin :
    integralClosure ℤ K = Algebra.adjoin ℤ ({cubeRootThree} : Set K)
```

**Theorem 5.10** (integralClosure_eq_adjoin)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
noncomputable def concreteIntegralBasis : Basis (Fin 3) ℤ (𝓞 K) :=
  Basis.mk linearIndependent_integralBasisVectorsInt
    (span_integralBasisVectorsInt_eq_top).ge
```

**Definition 5.11** (concreteIntegralBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

**Theorem 5.12** (ringOfIntegers_eq)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 5.13** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\sqrt[3]{3}]$, with basis $1, \theta, \theta^2$. The proof is more delicate than a quadratic case: a "peeling" argument uses the norm form to show an integral element with $3z \in \mathbb{Z}[\theta]$ already lies in $\mathbb{Z}[\theta]$.

## 5.3 Discriminant.lean

```
theorem numberField_discr_eq : NumberField.discr K = (-243 : ℤ)
```

**Theorem 5.14** (numberField_discr_eq)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem integralBasis_discr :
    Algebra.discr ℚ (NumberField.integralBasis K) = (-243 : ℚ)
```

**Theorem 5.15** (integralBasis_discr)**.** Mathematically, this identifies the displayed integral basis or compares it with mathlib's canonical integral basis.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 5.16** (Subsection note)**.** The discriminant is $-243 = -3^5$. The proof compares the concrete integral basis with mathlib's canonical integral basis.

## 5.4 NormForm.lean

```
theorem intNorm_eq_normForm (x y z : ℤ) :
    Algebra.norm ℤ (x • 1 + y • thetaInt + z • thetaInt ^ 2) =
      x ^ 3 + 3 * y ^ 3 + 9 * z ^ 3 - 9 * x * y * z
```

**Theorem 5.17** (intNorm_eq_normForm)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem isUnit_iff_normForm (x y : ℤ) :
    IsUnit (x • (1 : 𝓞 K) + y • thetaInt) ↔ x ^ 2 + 6 * y ^ 2 = 1
```

**Theorem 5.18** (isUnit_iff_normForm)**.** Mathematically, this converts the unit condition into the corresponding norm-form equation.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 5.19** (Subsection note)**.** This packages the cubic norm form for integral elements and the unit criterion.

## 5.5 PrimeSplitting.lean

```
theorem span_three_eq : Ideal.span {(3 :
    𝒪 K)} = Ideal.span {thetaInt} ^ 3
```

**Theorem 5.20** (span_three_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_two_eq :
    Ideal.span {(2 : 𝒪 K)} =
      Ideal.span {thetaInt - 1} *
          Ideal.span {thetaInt ^ 2 + thetaInt + 1}
```

**Theorem 5.21** (span_two_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_seven : (Ideal.span {(7 : 𝒪 K)}).IsPrime
```

**Theorem 5.22** (isPrime_span_seven)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
def primeAbove61 (r : ℤ) : Ideal (𝒪 K) :=
    Ideal.span {(61 : 𝒪 K), thetaInt - (r : 𝒪 K)}
```

**Definition 5.23** (primeAbove61)**.** This declaration introduces a concrete ideal used to describe splitting or ideal classes.

```
theorem span_sixtyOne_eq :
    Ideal.span {(61 : 𝒪 K)} = primeAbove61 4 * primeAbove61 5 *
        primeAbove61 52
```

**Theorem 5.24** (span_sixtyOne_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem exists_degree_one_prime_iff_cube_root_mod_p (p : ℕ) (hp :
    p.Prime)
    (hp3 : p ≠ 3) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
      Ideal.absNorm P = p) ↔ ∃ r : ZMod p, r ^ 3 = (3 : ZMod p)
```

**Theorem 5.25** (exists_degree_one_prime_iff_cube_root_mod_p)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem ramified_iff_eq_three (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p :
        𝒪 K)} ≤ P ^ 2) ↔ p = 3
```

**Theorem 5.26** (ramified_iff_eq_three)**.** Mathematically, this classifies exactly which rational primes ramify in the field.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 5.27** (Subsection note)**.** The prime 3 is totally ramified, 2 has one degree-one and one degree-two prime, 7 is inert, 61 splits into three degree-one primes, and degree-one primes above $p \neq 3$ are equivalent to 3 being a cube modulo $p$. Ramification occurs only at 3.

## 5.6 Units.lean

```
theorem torsionOrder_eq_two : NumberField.Units.torsionOrder K = 2
```

**Theorem 5.28** (torsionOrder_eq_two)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem not_totallyReal : ¬ NumberField.IsTotallyReal K
```

**Theorem 5.29** (not_totallyReal)**.** Mathematically, this computes the archimedean signature of the field.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem nrRealPlaces_eq_one : NumberField.InfinitePlace.nrRealPlaces K =
    1
```

**Theorem 5.30** (nrRealPlaces_eq_one)**.** Mathematically, this computes the archimedean signature of the field.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem nrComplexPlaces_eq_one :
    NumberField.InfinitePlace.nrComplexPlaces K = 1
```

**Theorem 5.31** (nrComplexPlaces_eq_one)**.** Mathematically, this computes the archimedean signature of the field.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := thetaInt ^ 2 - 2
  inv := 2 * thetaInt ^ 2 + 3 * thetaInt + 4
  val_inv := by have h := thetaInt_cube; linear_combination (2 *
      thetaInt + 3) * h
  inv_val := by have h := thetaInt_cube; linear_combination (2 *
      thetaInt + 3) * h
```

**Definition 5.32** (fundamentalUnit)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
      NumberField.Units.torsion K = ⊤
```

**Theorem 5.33** (fundamentalUnitSystem_isFundamental)**.** Mathematically, this states that the displayed unit generates the free part of the unit group, up to torsion.

*Proof sketch.* This declaration is currently stated in Lean with a remaining sorry; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

```
theorem regulator_eq :
    NumberField.Units.regulator K =
      |Real.log ((3 : ℝ) ^ ((2 : ℝ) / 3) - 2)|
```

**Theorem 5.34** (regulator_eq)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* This declaration is currently stated in Lean with a remaining sorry; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 5.35** (Subsection note)**.** The signature is $(1, 1)$, so the unit rank is 1, and the proposed unit is $\theta^2 - 2$. The concrete fundamental-unit statement and regulator formula are stated with sorry.

## 5.7 ClassGroup.lean

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 5.36** (classNumber_eq_one)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝓞 K)
```

**Theorem 5.37** (isPrincipalIdealRing)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝓞 K)
```

**Theorem 5.38** (uniqueFactorizationMonoid)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝓞 K) ≃* Multiplicative (ZMod 1))
```

**Theorem 5.39** (classGroupStructure)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 5.40** (Subsection note)**.** Class number one is proved using the Minkowski bound $(4/\pi)(3!/3^3)\sqrt{243} < 5$, reducing to primes $2, 3$, whose prime ideals were already explicitly principal.

## 5.8 Euclidean.lean

```
theorem normEuclidean (a b : 𝓞 K) (hb : b ≠ 0) :
    ∃ q r : 𝓞 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 5.41** (normEuclidean)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 5.42** (Subsection note)**.** This classical norm-Euclidean statement is present but still has `sorry`. The missing work is the geometry-of-numbers/nearest-lattice-point estimate for the cubic norm.

## 5.9 `Galois.lean`

```
theorem aut_eq_id (σ : K ≃ₐ[ℚ] K) : σ = idAut
```

**Theorem 5.43** (`aut_eq_id`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem not_isGalois : ¬ IsGalois ℚ K
```

**Theorem 5.44** (`not_isGalois`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
abbrev galoisClosure : Type := cubePolynomial.SplittingField
```

**Definition 5.45** (`galoisClosure`)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
theorem galoisClosureGroupStructure :
    Nonempty ((galoisClosure ≃ₐ[ℚ] galoisClosure) ≃* Equiv.Perm (Fin 3))
```

**Theorem 5.46** (`galoisClosureGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem finrank_galoisClosure : Module.finrank ℚ galoisClosure = 6
```

**Theorem 5.47** (`finrank_galoisClosure`)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 5.48** (Subsection note)**.** The cubic field itself has only the identity automorphism and is not Galois. Its splitting field has degree 6 and Galois group $S_3$.

### 5.10 Summary for $\mathbb{Q}(\sqrt[3]{3})$

Easy: basic `AdjoinRoot` construction. Hard but completed: ring of integers, prime splitting, non-Galois/Galois-closure structure, and class number one. Missing: norm-Euclideanity, concrete fundamental-unit generation, and regulator.

# 6 $\mathbb{Q}(\zeta_3)$

## 6.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 3 ℚ
```

**Definition 6.1** (`K`)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 3 ℚ K
```

**Definition 6.2** (`ζ`)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem zeta_sq_add_zeta_add_one_eq_zero : ζ ^ 2 + ζ + 1 = 0
```

**Theorem 6.3** (`zeta_sq_add_zeta_add_one_eq_zero`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem degree_eq_two : Module.finrank ℚ K = 2
```

**Theorem 6.4** (`degree_eq_two`)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
def sqrtNegThreeEquivCyclotomic : QSqrtNegThree ≃ₐ[ℚ] K :=
  sqrtNegThreeEquivZetaThree.trans adjoinRootEquivCyclotomic
```

**Definition 6.5** (`sqrtNegThreeEquivCyclotomic`)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

**Remark 6.6** (Subsection note)**.** This is the Eisenstein field, with $\zeta^2 + \zeta + 1 = 0$, degree 2, and an explicit equivalence with $\mathbb{Q}(\sqrt{-3})$.

## 6.2 RingOfIntegers.lean

```
noncomputable def zetaInt : 𝓞 K := zeta_isPrimitiveRoot.toInteger
```

**Definition 6.7** (zetaInt)**.** This declaration fixes a concrete object used by the following formal statements.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝓞 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

**Definition 6.8** (integralPowerBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
abbrev zetaExplicitIntegralBasis : Basis (Fin 2) ℤ (𝓞 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

**Definition 6.9** (zetaExplicitIntegralBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
theorem isIntegralClosure_adjoin_zeta :
    IsIntegralClosure (Algebra.adjoin ℤ ({ζ} : Set K)) ℤ K
```

**Theorem 6.10** (isIntegralClosure_adjoin_zeta)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 6.11** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\zeta_3]$, with basis $1, \zeta$. The proof is mostly provided by the cyclotomic integral power basis API.

## 6.3 Discriminant.lean

```
theorem numberField_discr_eq : NumberField.discr K = -3
```

**Theorem 6.12** (numberField_discr_eq)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem zetaExplicitIntegralBasis_discr :
    Algebra.discr ℤ zetaExplicitIntegralBasis = (-3 : ℤ)
```

**Theorem 6.13** (zetaExplicitIntegralBasis_discr)**.** Mathematically, this identifies the displayed integral basis or compares it with mathlib's canonical integral basis.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
example
    (P : Matrix (Free.ChooseBasisIndex ℤ (𝓞 K)) (Free.ChooseBasisIndex ℤ
        (𝓞 K)) ℚ) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
        (NumberField.integralBasis K)) =
      P.det ^ 2 * Algebra.discr ℚ (NumberField.integralBasis K)
```

**Theorem 6.14** (`example`)**.** Mathematically, replacing the canonical integral basis by a rational matrix multiple scales the discriminant by $\det(P)^2$.

*Proof sketch.* The Lean proof applies `Algebra.discr_of_matrix_mulVec`. □

```
example
    (P : Matrix (Free.ChooseBasisIndex ℤ (𝓞 K)) (Free.ChooseBasisIndex ℤ
        (𝓞 K)) ℚ)
    (hdet : P.det = 1 ∨ P.det = -1) :
    Algebra.discr ℚ (Matrix.mulVec (P.map (algebraMap ℚ K))
        (NumberField.integralBasis K)) =
        (-3 : ℚ)
```

**Theorem 6.15** (`example`)**.** Mathematically, a unimodular change of basis preserves the Eisenstein-field discriminant $-3$.

*Proof sketch.* The Lean proof reduces to the matrix discriminant formula and `integralBasis_discr`. □

**Remark 6.16** (Subsection note)**.** The discriminant is $-3$, obtained from the explicit cyclotomic basis.

## 6.4 `PrimeSplitting.lean`

```
theorem intNorm_eq_normForm (x y : ℤ) :
    Algebra.norm ℤ (x • (1 : 𝓞 K) + y • zetaInt) = x ^ 2 - x * y + y ^ 2
```

**Theorem 6.17** (`intNorm_eq_normForm`)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem span_three_eq :
    Ideal.span {(3 : 𝓞 K)} = Ideal.span {zetaInt - 1} ^ 2
```

**Theorem 6.18** (span_three_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝒪 K)}).IsPrime
```

**Theorem 6.19** (isPrime_span_two)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_seven_eq :
    Ideal.span {(7 : 𝒪 K)} =
      Ideal.span {3 + zetaInt} * Ideal.span {3 + zetaInt ^ 2}
```

**Theorem 6.20** (span_seven_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem splits_iff_one_mod_three (p : ℕ) (hp : p.Prime) (hp3 : p ≠ 3) :
    (∃ P Q : Ideal (𝒪 K), P.IsPrime ∧ Q.IsPrime ∧ P ≠ Q ∧
      Ideal.span {(p : 𝒪 K)} = P * Q) ↔ p % 3 = 1
```

**Theorem 6.21** (splits_iff_one_mod_three)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 6.22** (Subsection note)**.** The norm form is $x^2 - xy + y^2$. The prime 3 ramifies, 2 is inert, 7 splits, and $p \neq 3$ splits iff $p \equiv 1 \pmod 3$.

## 6.5 Units.lean

```
abbrev etaUnit : (𝒪 K)ˣ :=
  (zeta_isPrimitiveRoot.toInteger_isPrimitiveRoot.isUnit (by
      decide)).unit
```

**Definition 6.23** (etaUnit)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem unit_eq_one_or_neg_one_or_eta (u : (𝓞 K)ˣ) :
    u = 1 ∨ u = -1 ∨ u = etaUnit ∨ u = -etaUnit ∨
    u = etaUnit ^ 2 ∨ u = -(etaUnit ^ 2)
```

**Theorem 6.24** (`unit_eq_one_or_neg_one_or_eta`)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem torsionOrder_eq_six : Units.torsionOrder K = 6
```

**Theorem 6.25** (`torsionOrder_eq_six`)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_zero : Units.rank K = 0
```

**Theorem 6.26** (`unitRank_eq_zero`)**.** Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitGroupStructure : Nonempty ((𝓞 K)ˣ ≃* Multiplicative (ZMod
    6))
```

**Theorem 6.27** (`unitGroupStructure`)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem regulator_eq_one : Units.regulator K = 1
```

**Theorem 6.28** (`regulator_eq_one`)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

**Remark 6.29** (Subsection note)**.** All units are the six roots of unity. Since the rank is 0, the regulator is 1.

## 6.6 ClassGroup.lean

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝓞 K)
```

**Theorem 6.30** (isPrincipalIdealRing)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝓞 K)
```

**Theorem 6.31** (uniqueFactorizationMonoid)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 6.32** (classNumber_eq_one)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem classGroupStructure : Nonempty (ClassGroup (𝓞 K) ≃*
    Multiplicative (ZMod 1))
```

**Theorem 6.33** (classGroupStructure)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 6.34** (Subsection note)**.** The Eisenstein integers have class number one.

## 6.7 Euclidean.lean

```
theorem normEuclidean (a b : 𝓞 K) (hb : b ≠ 0) :
    ∃ q r : 𝓞 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 6.35** (normEuclidean)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* The proof chooses a nearest integral-coordinate quotient and bounds the norm of the remainder; in the unfinished higher-degree cases this is exactly the missing geometry-of-numbers step. □

**Remark 6.36** (Subsection note)**.** This is the Euclidean algorithm for Eisenstein integers, using coordinate rounding and the hexagonal norm bound $\leq 3/4$.

### 6.8 `Galois.lean`

```
theorem isGalois : IsGalois ℚ K
```

**Theorem 6.37** (`isGalois`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
abbrev galEquivZMod : Gal(K / ℚ) ≃* (ZMod 3)ˣ :=
  IsCyclotomicExtension.Rat.galEquivZMod 3 K
```

**Definition 6.38** (`galEquivZMod`)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

```
theorem galoisGroupStructure :
    Nonempty (Gal(K/ℚ) ≃* Multiplicative (ZMod 2))
```

**Theorem 6.39** (`galoisGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 6.40** (Subsection note)**.** The Galois group is $(\mathbb{Z}/3\mathbb{Z})^\times \cong C_2$.

### 6.9 Summary for $\mathbb{Q}(\zeta_3)$

This is the smoothest example: cyclotomic APIs directly give the field, integral basis, Galois group, finite units, and much of the class group structure. The harder completed part is the Euclidean proof and prime splitting criterion.

## 7 $\mathbb{Q}(\zeta_5)$

### 7.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 5 ℚ
```

**Definition 7.1** (K)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 5 ℚ K
```

**Definition 7.2** (ζ)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem zeta_pow_five : ζ ^ 5 = 1
```

**Theorem 7.3** (zeta_pow_five)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem zeta_sum_eq_zero : 1 + ζ + ζ ^ 2 + ζ ^ 3 + ζ ^ 4 = 0
```

**Theorem 7.4** (zeta_sum_eq_zero)**.** Mathematically, this is one of the concrete arithmetic statements recorded in this file.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem degree_eq_four : Module.finrank ℚ K = 4
```

**Theorem 7.5** (degree_eq_four)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem zetaFivePolynomial_eq_minpoly :
    zetaFivePolynomial = minpoly ℚ ζ
```

**Theorem 7.6** (zetaFivePolynomial_eq_minpoly)**.** Mathematically, this identifies the minimal polynomial of the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

**Remark 7.7** (Subsection note)**.** The field is $\mathbb{Q}(\zeta_5)$, degree 4, with minimal polynomial $\Phi_5 = X^4 + X^3 + X^2 + X + 1$.

## 7.2 RingOfIntegers.lean

```
theorem zeta_integral : ζ ∈ integralClosure ℤ K
```

**Theorem 7.8** (zeta_integral)**.** Mathematically, this proves that the displayed algebraic element is integral over $\mathbb{Z}$.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
noncomputable def zetaInt : 𝓞 K := ⟨ζ, zeta_integral⟩
```

**Definition 7.9** (zetaInt)**.** This declaration fixes a concrete object used by the following formal statements.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝓞 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

**Definition 7.10** (integralPowerBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
def integralBasisVectors : Fin 4 → K := ![1, ζ, ζ ^ 2, ζ ^ 3]
```

**Definition 7.11** (integralBasisVectors)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
noncomputable def concreteIntegralBasis : Basis (Fin 4) ℤ (𝓞 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

**Definition 7.12** (concreteIntegralBasis)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

**Theorem 7.13** (ringOfIntegers_eq)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 7.14** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\zeta_5]$, with basis $1, \zeta, \zeta^2, \zeta^3$. This is again largely powered by the cyclotomic integral basis API.

## 7.3 Discriminant.lean

```
theorem numberField_discr_eq : NumberField.discr K = (125 : ℤ)
```

**Theorem 7.15** (numberField_discr_eq)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (125 : ℤ)
```

**Theorem 7.16** (concreteIntegralBasis_discr)**.** Mathematically, this identifies the displayed integral basis or compares it with mathlib's canonical integral basis.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 7.17** (Subsection note)**.** The discriminant is $5^3 = 125$.

## 7.4 PrimeSplitting.lean

```
def normForm {R : Type*} [CommRing R] (a b c d : R) : R :=
  a ^ 4 - a ^ 3 * b - a ^ 3 * c - a ^ 3 * d +
    a ^ 2 * b ^ 2 + 2 * a ^ 2 * b * c + 2 * a ^ 2 * b * d +
    a ^ 2 * c ^ 2 - 3 * a ^ 2 * c * d + a ^ 2 * d ^ 2 -
    a * b ^ 3 - 3 * a * b ^ 2 * c + 2 * a * b ^ 2 * d +
    2 * a * b * c ^ 2 - a * b * c * d - 3 * a * b * d ^ 2 -
    a * c ^ 3 + 2 * a * c ^ 2 * d + 2 * a * c * d ^ 2 - a * d ^ 3 +
    b ^ 4 - b ^ 3 * c - b ^ 3 * d + b ^ 2 * c ^ 2 +
    2 * b ^ 2 * c * d + b ^ 2 * d ^ 2 - b * c ^ 3 -
    3 * b * c ^ 2 * d + 2 * b * c * d ^ 2 - b * d ^ 3 +
    c ^ 4 - c ^ 3 * d + c ^ 2 * d ^ 2 - c * d ^ 3 + d ^ 4
```

**Definition 7.18** (normForm)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
theorem intNorm_eq_normForm (a b c d : ℤ) :
    Algebra.norm ℤ (a + bζ + cζ^2 + dζ^3) = normForm a b c d
```

**Theorem 7.19** (intNorm_eq_normForm)**.** Mathematically, this computes the field norm or integral norm as an explicit polynomial form.

*Proof sketch.* The proof expands the algebra norm in the chosen basis and then simplifies the resulting polynomial identity. □

```
theorem span_five_eq :
    Ideal.span {(5 : 𝓞 K)} = Ideal.span {1 - zetaInt} ^ 4
```

**Theorem 7.20** (span_five_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem isPrime_span_two : (Ideal.span {(2 : 𝓞 K)}).IsPrime
```

**Theorem 7.21** (isPrime_span_two)**.** Mathematically, this proves that the displayed ideal or element is prime.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_nineteen_eq :
    Ideal.span {(19 : 𝓞 K)} =
      Ideal.span {(19 : 𝓞 K), zetaInt ^ 2 + 5 * zetaInt + 1} *
        Ideal.span {(19 : 𝓞 K), zetaInt ^ 2 + 15 * zetaInt + 1}
```

**Theorem 7.22** (span_nineteen_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_eleven_eq :
    Ideal.span {(11 : 𝓞 K)} =
      Ideal.span {(11 : 𝓞 K), zetaInt - 3} * Ideal.span {(11 :
          𝓞 K), zetaInt - 4} *
        Ideal.span {(11 : 𝓞 K), zetaInt - 5} * Ideal.span {(11 :
            𝓞 K), zetaInt - 9}
```

**Theorem 7.23** (span_eleven_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem splits_completely_iff (p : ℕ) (hp : p.Prime) (hp5 : p ≠ 5) :
    (∃ P : Ideal (𝓞 K), P.IsPrime ∧ Ideal.span {(p : 𝓞 K)} ≤ P ∧
      Ideal.absNorm P = p) ↔
      p % 5 = 1
```

**Theorem 7.24** (`splits_completely_iff`)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem ramified_iff_eq_five (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p :
        𝒪 K)} ≤ P ^ 2) ↔ p = 5
```

**Theorem 7.25** (`ramified_iff_eq_five`)**.** Mathematically, this classifies exactly which rational primes ramify in the field.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 7.26** (Subsection note)**.** The prime 5 totally ramifies, 2 is inert of residue degree 4, 19 factors into two degree-two primes, and 11 splits completely. In general, $p \neq 5$ splits completely iff $p \equiv 1 \pmod 5$. The proof has a long explicit quartic norm-form and kernel-of-residue-map computations.

## 7.5 `Units.lean`

```
noncomputable def torsionGenerator : (𝒪 K)ˣ where
  val := -zetaInt
  inv := -zetaInt ^ 4
  val_inv := by linear_combination zetaInt_pow_five
  inv_val := by linear_combination zetaInt_pow_five
```

**Definition 7.27** (`torsionGenerator`)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem torsionOrder_eq_ten : NumberField.Units.torsionOrder K = 10
```

**Theorem 7.28** (`torsionOrder_eq_ten`)**.** Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_one : NumberField.Units.rank K = 1
```

**Theorem 7.29** (`unitRank_eq_one`)**.** Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
noncomputable def fundamentalUnit : (𝓞 K)ˣ where
  val := 1 + zetaInt
  inv := -(zetaInt + zetaInt ^ 3)
  val_inv := by linear_combination -zetaInt_sum_eq_zero
  inv_val := by linear_combination -zetaInt_sum_eq_zero
```

**Definition 7.30** (`fundamentalUnit`)**.** This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
      NumberField.Units.torsion K = ⊤
```

**Theorem 7.31** (`fundamentalUnitSystem_isFundamental`)**.** Mathematically, this states that the displayed unit generates the free part of the unit group, up to torsion.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

```
theorem regulator_eq :
    NumberField.Units.regulator K = 2 * Real.log ((1 + Real.sqrt 5) / 2)
```

**Theorem 7.32** (`regulator_eq`)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 7.33** (Subsection note)**.** The torsion subgroup is $\mu_{10}$, generated by $-\zeta$, and the rank is 1. The proposed free generator is the cyclotomic unit $1+\zeta$. Its concrete fundamental-unit theorem and regulator formula are stated with `sorry`.

## 7.6 `ClassGroup.lean`

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 7.34** (`classNumber_eq_one`)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝓞 K)
```

**Theorem 7.35** (`isPrincipalIdealRing`)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝓞 K)
```

**Theorem 7.36** (`uniqueFactorizationMonoid`)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝓞 K) ≃* Multiplicative (ZMod 1))
```

**Theorem 7.37** (`classGroupStructure`)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 7.38** (Subsection note)**.** Class number one follows from a Minkowski-bound reduction to small primes plus the explicit splitting/principal ideal computations.

## 7.7 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝓞 K) (hb : b ≠ 0) :
    ∃ q r : 𝓞 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 7.39** (`normEuclidean`)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 7.40** (Subsection note)**.** This norm-Euclidean statement is present with `sorry`. The missing piece is a quartic lattice rounding/norm estimate.

## 7.8 `QuadraticSubfield.lean`

```
theorem gaussSum_sq : (1 + 2 * ζ + 2 * ζ ^ 4) ^ 2 = 5
```

**Theorem 7.41** (`gaussSum_sq`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
abbrev sqrtFiveSubfield : Subalgebra ℚ K :=
    Algebra.adjoin ℚ ({gaussSum} : Set K)
```

**Definition 7.42** (`sqrtFiveSubfield`). This declaration identifies the relevant subfield generated by the displayed element.

```
noncomputable def sqrtFiveToCyclotomic : QSqrtFive →ₐ[ℚ] K := by
  exact AdjoinRoot.liftAlgHom sqrtFivePolynomial (Algebra.ofId ℚ K)
      gaussSum (by
    simp [sqrtFivePolynomial, gaussSum, gaussSum_sq])
```

**Definition 7.43** (`sqrtFiveToCyclotomic`). This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

```
noncomputable def sqrtFiveEquivSubfield :
    QSqrtFive ≃ₐ[ℚ] sqrtFiveSubfield := by
  let gs : sqrtFiveSubfield :=
      ⟨gaussSum, Algebra.self_mem_adjoin_singleton ℚ gaussSum⟩
  let g : QSqrtFive →ₐ[ℚ] sqrtFiveSubfield :=
    AdjoinRoot.liftAlgHom sqrtFivePolynomial (Algebra.ofId ℚ
        sqrtFiveSubfield) gs (by
      ext
      simp [sqrtFivePolynomial, gs, gaussSum, gaussSum_sq])
  refine AlgEquiv.ofBijective g ⟨RingHom.injective g.toRingHom, ?_⟩
  intro y
  let p : (x : K) → x ∈ Algebra.adjoin ℚ ({gaussSum} : Set K) → Prop :=
    fun x hx => ∃ z : QSqrtFive, g z = ⟨x, hx⟩
  change p y.1 y.2
  refine Algebra.adjoin_induction (s :=
      ({gaussSum} : Set K)) ?_ ?_ ?_ ?_ y.2
  · intro x hx
    rcases hx with rfl
    exact ⟨sqrtFiveRoot,
      AdjoinRoot.liftAlgHom_root sqrtFivePolynomial (Algebra.ofId ℚ
          sqrtFiveSubfield) gs _⟩
  · intro r
    exact ⟨algebraMap ℚ QSqrtFive r, g.commutes r⟩
  · intro x y hx hy hx' hy'
    rcases hx' with ⟨x', hxeq⟩
    rcases hy' with ⟨y', hyeq⟩
```

```
    refine ⟨x' + y', ?_⟩
    rw [map_add, hxeq, hyeq]
    ext
    rfl
  · intro x y hx hy hx' hy'
    rcases hx' with ⟨x', hxeq⟩
    rcases hy' with ⟨y', hyeq⟩
    refine ⟨x' * y', ?_⟩
    rw [map_mul, hxeq, hyeq]
    ext
    rfl
```

**Definition 7.44** (`sqrtFiveEquivSubfield`)**.** This declaration constructs a concrete algebra homomorphism or equivalence between two presentations.

**Remark 7.45** (Subsection note)**.** The quadratic subfield is $\mathbb{Q}(\sqrt{5})$, generated by the quadratic Gauss sum $1 + 2\zeta + 2\zeta^4$.

## 7.9 `Galois.lean`

```
noncomputable def sigmaTwo : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
      2 (by norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

**Definition 7.46** (`sigmaTwo`)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem sigmaTwo_apply_zeta : sigmaTwo ζ = ζ ^ 2
```

**Theorem 7.47** (`sigmaTwo_apply_zeta`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroupStructure : Nonempty ((K ≃ₐ[ℚ] K) ≃* (ZMod 5)ˣ)
```

**Theorem 7.48** (`galoisGroupStructure`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroup_isCyclic : IsCyclic (K ≃ₐ[ℚ] K)
```

**Theorem 7.49** (`galoisGroup_isCyclic`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem orderOf_sigmaTwo : orderOf sigmaTwo = 4
```

**Theorem 7.50** (`orderOf_sigmaTwo`)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 7.51** (Subsection note)**.** The Galois group is $(\mathbb{Z}/5\mathbb{Z})^\times \cong C_4$, generated by $\zeta \mapsto \zeta^2$.

### 7.10 Summary for $\mathbb{Q}(\zeta_5)$

Easy: cyclotomic field, integral basis, Galois group. Hard but completed: explicit quartic norm form, prime splitting, class number one, quadratic subfield. Missing: norm-Euclideanity, concrete cyclotomic-unit generation, and regulator.

# 8 $\mathbb{Q}(\zeta_8)$

## 8.1 `Basic.lean`

```
abbrev K : Type := CyclotomicField 8 ℚ
```

**Definition 8.1** (`K`)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
abbrev ζ : K := IsCyclotomicExtension.zeta 8 ℚ K
```

**Definition 8.2** (`ζ`)**.** This declaration names the distinguished algebraic generator of the field.

```
theorem zeta_pow_eight : ζ ^ 8 = 1
```

**Theorem 8.3** (`zeta_pow_eight`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem zeta_pow_four : ζ ^ 4 = -1
```

**Theorem 8.4** (`zeta_pow_four`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem degree_eq_four : Module.finrank ℚ K = 4
```

**Theorem 8.5** (`degree_eq_four`)**.** Mathematically, this states the degree of the number field over $\mathbb{Q}$.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem zetaEightPolynomial_eq_minpoly :
    zetaEightPolynomial = minpoly ℚ ζ
```

**Theorem 8.6** (`zetaEightPolynomial_eq_minpoly`)**.** Mathematically, this identifies the minimal polynomial of the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

**Remark 8.7** (Subsection note)**.** The field is $\mathbb{Q}(\zeta_8)$, degree 4, with minimal polynomial $X^4 + 1$.

## 8.2 `RingOfIntegers.lean`

```
noncomputable def zetaInt : 𝓞 K := ⟨ζ, zeta_integral⟩
```

**Definition 8.8** (`zetaInt`)**.** This declaration fixes a concrete object used by the following formal statements.

```
abbrev integralPowerBasis : PowerBasis ℤ (𝓞 K) :=
  zeta_isPrimitiveRoot.integralPowerBasis
```

**Definition 8.9** (`integralPowerBasis`)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
def integralBasisVectors : Fin 4 → K := ![1, ζ, ζ ^ 2, ζ ^ 3]
```

**Definition 8.10** (`integralBasisVectors`)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
noncomputable def concreteIntegralBasis : Basis (Fin 4) ℤ (𝓞 K) :=
  integralPowerBasis.basis.reindex (finCongr integralPowerBasis_dim)
```

**Definition 8.11** (`concreteIntegralBasis`)**.** This declaration gives the explicit basis used for coordinates, discriminants, norms, or the ring of integers.

```
theorem ringOfIntegers_eq : integralClosure ℤ K = concreteRingOfIntegers
```

**Theorem 8.12** (`ringOfIntegers_eq`)**.** Mathematically, this identifies the full ring of integers with the displayed generated order.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 8.13** (Subsection note)**.** The ring of integers is $\mathbb{Z}[\zeta_8]$, with the standard cyclotomic basis.

## 8.3 `Discriminant.lean`

```
theorem numberField_discr_eq : NumberField.discr K = (256 : ℤ)
```

**Theorem 8.14** (`numberField_discr_eq`)**.** Mathematically, this computes the discriminant in the displayed basis or for the number field.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

```
theorem concreteIntegralBasis_discr :
    Algebra.discr ℤ concreteIntegralBasis = (256 : ℤ)
```

**Theorem 8.15** (`concreteIntegralBasis_discr`)**.** Mathematically, this identifies the displayed integral basis or compares it with mathlib's canonical integral basis.

*Proof sketch.* The proof uses integrality of the generator, the explicit basis, and discriminant/norm computations to compare the generated order with the integral closure. □

**Remark 8.16** (Subsection note)**.** The discriminant is 256.

## 8.4 `Subfields.lean`

```
theorem sqrt_two_sq : (ζ + ζ ^ 7) ^ 2 = 2
```

**Theorem 8.17** (`sqrt_two_sq`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem i_sq : (ζ ^ 2) ^ 2 = -1
```

**Theorem 8.18** (`i_sq`)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

```
theorem sqrt_neg_two_sq : (ζ - ζ ^ 7) ^ 2 = -2
```

**Theorem 8.19** (sqrt_neg_two_sq)**.** Mathematically, this records the defining algebraic relation satisfied by the chosen generator.

*Proof sketch.* The proof is a direct calculation from the chosen presentation, irreducibility, or the power-basis API. □

**Remark 8.20** (Subsection note)**.** The three quadratic directions inside $\mathbb{Q}(\zeta_8)$ are visible: $\sqrt{2}$, $i$, and $\sqrt{-2}$.

## 8.5 PrimeSplitting.lean

```
def normForm {R : Type*} [CommRing R] (a b c d : R) : R :=
  (a ^ 2 - c ^ 2 + 2 * b * d) ^ 2 + (2 * a * c - b ^ 2 + d ^ 2) ^ 2
```

**Definition 8.21** (normForm)**.** This declaration fixes the Lean model for the field or an auxiliary presentation used in this file.

```
theorem span_two_eq :
    Ideal.span {(2 : 𝓞 K)} = Ideal.span {1 - zetaInt} ^ 4
```

**Theorem 8.22** (span_two_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_three_eq :
    Ideal.span {(3 : 𝓞 K)} =
      Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + zetaInt + 2} *
        Ideal.span {(3 : 𝓞 K), zetaInt ^ 2 + 2 * zetaInt + 2}
```

**Theorem 8.23** (span_three_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem span_seventeen_eq :
    Ideal.span {(17 : 𝓞 K)} =
      Ideal.span {(17 : 𝓞 K), zetaInt - 2} * Ideal.span {(17 :
```

```
    𝒪 K), zetaInt - 8} *
  Ideal.span {(17 : 𝒪 K), zetaInt - 9} * Ideal.span {(17 :
    𝒪 K), zetaInt - 15}
```

**Theorem 8.24** (span_seventeen_eq)**.** Mathematically, this gives an explicit factorization of a rational prime ideal in the ring of integers.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem no_inert_prime (p : ℕ) (hp : p.Prime) :
    ¬ (Ideal.span {(p : 𝒪 K)}).IsPrime
```

**Theorem 8.25** (no_inert_prime)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem splits_completely_iff (p : ℕ) (hp : p.Prime) (hp2 : p ≠ 2) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p : 𝒪 K)} ≤ P ∧
      Ideal.absNorm P = p) ↔
      p % 8 = 1
```

**Theorem 8.26** (splits_completely_iff)**.** Mathematically, this gives a congruence or residue-field criterion for splitting behavior.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

```
theorem ramified_iff_eq_two (p : ℕ) (hp : p.Prime) :
    (∃ P : Ideal (𝒪 K), P.IsPrime ∧ Ideal.span {(p :
        𝒪 K)} ≤ P ^ 2) ↔ p = 2
```

**Theorem 8.27** (ramified_iff_eq_two)**.** Mathematically, this classifies exactly which rational primes ramify in the field.

*Proof sketch.* The proof expands the relevant ideal generators, computes absolute norms, and uses residue or norm comparisons to prove primality, ramification, and splitting. □

**Remark 8.28** (Subsection note)**.** The prime 2 totally ramifies, 3 splits into two degree-two primes, 17 splits completely, no rational prime is inert, complete splitting is $p \equiv 1 \pmod 8$, and ramification occurs only at 2.

## 8.6 Units.lean

```
noncomputable def torsionGenerator : (𝒪 K)ˣ where
  val := zetaInt
  inv := zetaInt ^ 7
  val_inv := by linear_combination (zetaInt ^ 4 - 1) * zetaInt_pow_four
  inv_val := by linear_combination (zetaInt ^ 4 - 1) * zetaInt_pow_four
```

**Definition 8.29** (`torsionGenerator`). This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem torsionOrder_eq_eight : NumberField.Units.torsionOrder K = 8
```

**Theorem 8.30** (`torsionOrder_eq_eight`). Mathematically, this describes the roots of unity in the unit group.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
theorem unitRank_eq_one : NumberField.Units.rank K = 1
```

**Theorem 8.31** (`unitRank_eq_one`). Mathematically, this is Dirichlet's unit-rank formula specialized to this field.

*Proof sketch.* The proof combines the explicit norm form, the archimedean signature, and Dirichlet-unit infrastructure; concrete generators require the displayed Pell or cyclotomic-unit argument. □

```
noncomputable def fundamentalUnit : (𝒪 K)ˣ where
  val := 1 + zetaInt + zetaInt ^ 7
  inv := zetaInt + zetaInt ^ 7 - 1
  val_inv := by
    linear_combination (zetaInt ^ 10 - zetaInt ^ 6 + 2 *
        zetaInt ^ 4 + zetaInt ^ 2 - 2) *
      zetaInt_pow_four
  inv_val := by
    linear_combination (zetaInt ^ 10 - zetaInt ^ 6 + 2 *
        zetaInt ^ 4 + zetaInt ^ 2 - 2) *
      zetaInt_pow_four
```

**Definition 8.32** (`fundamentalUnit`). This declaration introduces a concrete unit or a concrete family of torsion units.

```
theorem fundamentalUnitSystem_isFundamental :
    Subgroup.closure (Set.range fundamentalUnitSystem) ⊔
      NumberField.Units.torsion K = ⊤
```

**Theorem 8.33** (`fundamentalUnitSystem_isFundamental`)**.** Mathematically, this states that the displayed unit generates the free part of the unit group, up to torsion.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

```
theorem regulator_eq :
    NumberField.Units.regulator K = 2 * Real.log (1 + Real.sqrt 2)
```

**Theorem 8.34** (`regulator_eq`)**.** Mathematically, this evaluates the regulator as the logarithmic covolume of the unit lattice.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 8.35** (Subsection note)**.** The torsion subgroup is $\mu_8$, the rank is 1, and the proposed generator is $1+\sqrt{2} = 1+\zeta+\zeta^7$. The concrete generation statement and regulator formula have `sorry`.

## 8.7 `ClassGroup.lean`

```
theorem classNumber_eq_one : NumberField.classNumber K = 1
```

**Theorem 8.36** (`classNumber_eq_one`)**.** Mathematically, this computes the class number of the ring of integers.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

```
theorem isPrincipalIdealRing : IsPrincipalIdealRing (𝓞 K)
```

**Theorem 8.37** (`isPrincipalIdealRing`)**.** Mathematically, this says the ring of integers is a PID, or in the negative case not a PID.

*Proof sketch.* The Lean proof follows the preceding concrete definitions and reduces the claim to explicit algebraic calculation or a standard mathlib API lemma. □

```
theorem uniqueFactorizationMonoid : UniqueFactorizationMonoid (𝓞 K)
```

**Theorem 8.38** (`uniqueFactorizationMonoid`)**.** Mathematically, this says the ring of integers is a unique factorization monoid.

*Proof sketch.* The proof uses the preceding PID statement as a local instance, then applies mathlib's PID-to-UFD instance. □

```
theorem classGroupStructure :
    Nonempty (ClassGroup (𝓞 K) ≃* Multiplicative (ZMod 1))
```

**Theorem 8.39** (`classGroupStructure`)**.** Mathematically, this identifies the ideal class group as the displayed finite cyclic group.

*Proof sketch.* The proof uses Minkowski's bound to reduce to finitely many small ideals, then uses the explicit splitting computations above to identify their ideal classes. □

**Remark 8.40** (Subsection note)**.** The proof uses Minkowski's bound; only 2 lies below the bound, and the ramified prime above 2 is principal.

## 8.8 `Euclidean.lean`

```
theorem normEuclidean (a b : 𝒪 K) (hb : b ≠ 0) :
    ∃ q r : 𝒪 K, a = b * q + r ∧
      (Algebra.norm ℤ r).natAbs < (Algebra.norm ℤ b).natAbs
```

**Theorem 8.41** (`normEuclidean`)**.** Mathematically, this states the norm-Euclidean division property, or its failure in the negative case.

*Proof sketch.* This declaration is currently stated in Lean with a remaining `sorry`; the mathematical proof would require the concrete unit or Euclidean estimate described here. □

**Remark 8.42** (Subsection note)**.** This norm-Euclidean statement is present with `sorry`.

## 8.9 `Galois.lean`

```
noncomputable def sigmaThree : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
      3 (by norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

**Definition 8.43** (`sigmaThree`)**.** This declaration fixes a concrete object used by the following formal statements.

```
noncomputable def sigmaFive : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
      5 (by norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

**Definition 8.44** (`sigmaFive`)**.** This declaration fixes a concrete object used by the following formal statements.

```
noncomputable def sigmaSeven : K ≃ₐ[ℚ] K :=
  IsCyclotomicExtension.fromZetaAut (zeta_isPrimitiveRoot.pow_of_coprime
      7 (by norm_num))
    (Polynomial.cyclotomic.irreducible_rat (by norm_num))
```

**Definition 8.45** (sigmaSeven)**.** This declaration fixes a concrete object used by the following formal statements.

```
theorem galoisGroupStructure : Nonempty ((K ≃ₐ[ℚ] K) ≃* (ZMod 8)ˣ)
```

**Theorem 8.46** (galoisGroupStructure)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroup_klein :
    Nonempty ((K ≃ₐ[ℚ] K) ≃*
      (Multiplicative (ZMod 2) × Multiplicative (ZMod 2)))
```

**Theorem 8.47** (galoisGroup_klein)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

```
theorem galoisGroup_not_isCyclic : ¬ IsCyclic (K ≃ₐ[ℚ] K)
```

**Theorem 8.48** (galoisGroup_not_isCyclic)**.** Mathematically, this identifies the relevant automorphism or the Galois group structure.

*Proof sketch.* The proof evaluates automorphisms on the generator and uses the defining polynomial to show that the listed automorphisms exhaust the group. □

**Remark 8.49** (Subsection note)**.** The Galois group is $(\mathbb{Z}/8\mathbb{Z})^\times \cong C_2 \times C_2$, not cyclic; every automorphism squares to the identity.

### 8.10 Summary for $\mathbb{Q}(\zeta_8)$

Easy: cyclotomic structure, integral basis, discriminant, subfield identities. Hard but completed: prime splitting, no-inert-prime theorem, class number one, Klein-four Galois structure. Missing: norm-Euclideanity, concrete unit generation, and regulator.

## 9 Final summary: current mathlib number field API

The current API is already strong for low-degree explicit fields. It supports: cyclotomic fields and primitive roots, AdjoinRoot models, integral closures and integral power bases, algebra norms and discriminants, ideals with absolute norm, ramification/inertia, class groups, Minkowski-bound class number proofs, and Galois groups.

The most successful patterns in this folder are:

- Cyclotomic ring-of-integers proofs: `IsCyclotomicExtension` gives very direct integral bases.
- Quadratic and small cubic norm-form computations: once the basis is explicit, many ideal and unit facts reduce to arithmetic.
- Class group proofs: `exists_ideal_in_class_of_norm_le` plus explicit small-prime splitting is powerful enough for class number 1 and 2 examples.
- Galois groups: cyclotomic Galois groups and quadratic conjugation are well supported.

The main rough edges are:

- Concrete Dirichlet units: mathlib has abstract `fundSystem` APIs, but proving a displayed unit is the chosen generator still needs bespoke Pell/cyclotomic-unit work.
- Regulators: comparing concrete logarithmic embeddings with mathlib's regulator API, especially `Units.regulator`, is not yet ergonomic.
- Norm-Euclidean proofs in degree 3 and 4: the missing part is mostly reusable geometry-of-numbers or lattice-rounding infrastructure.
- Prime splitting proofs are repetitive: many files rebuild residue maps and kernel computations that should be abstracted into a reusable "factor minimal polynomial mod $p$ gives ideal factorization" API.

Promising optimization routes:

- Build a reusable residue-hom/kernel lemma for $\mathbb{Z}[\alpha]/(p, f_i(\alpha))$.
- Add automation for norm forms from a `PowerBasis` or `Basis`.
- Package Minkowski-bound class-number-one proofs into a theorem requiring only the finite list of small principal prime ideals.
- Strengthen concrete unit APIs: Pell variants for nonmaximal-looking coordinates, cyclotomic units, and regulator evaluations.
- Create Euclidean templates for quadratic, pure cubic, and quartic cyclotomic lattices.